

Guida Lab 3D — Networking Base

Guida Lab — Modulo 3D: Networking di Base

Corso: Lab Amministrazione di Sistemi T **Fonte:**

LAB_Configurazione_della_rete_delle_VM_21-28_aprile.html (sezioni 1–2) [fonte: lab HTML] **Lezione teorica di riferimento:**

lezione_modulo3D_networking_base.md **Prerequisiti:** 3A (systemd — i servizi di rete sono demoni), 3B (apt — qui si installano tcpdump e tracepath)

Scope: questa guida copre gli strumenti di diagnostica di rete su **VM singola** (Es. 1–6). La configurazione multi-machine (Client-Router-Server) è in 3E.

Setup

```
cd ~/Progetti/sysAdmin-lab
vagrant up --provider=virtualbox
vagrant ssh
```

Prima di iniziare: verifica di essere dentro la VM.

```
hostname      # bookworm
whoami        # vagrant
```

Se `vagrant up` fallisce → controlla `~/UniCode/troubleshooting_vm.md`.

Es. 1 — Stato della rete: ip a e ip r

Comando

```
ip a
```

Cosa fa: mostra tutte le interfacce di rete con i loro indirizzi. ip è lo strumento moderno per gestire indirizzi, routing e link (sostituisce ifconfig). a è abbreviazione di address.

Output atteso:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 ...
   inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
   link/ether 08:00:27:XX:XX:XX brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic eth0
```

Cosa leggere: - lo = loopback (127.0.0.1) — interfaccia virtuale, traffico locale alla macchina - eth0 = interfaccia NAT di VirtualBox — 10.0.2.15/24 - link/ether = indirizzo MAC dell'interfaccia - dynamic = IP assegnato via DHCP (non statico)

```
# Mostra solo eth0
ip a show eth0
```

```
# Mostra la tabella di routing
ip r
```

Output ip r atteso:

```
default via 10.0.2.2 dev eth0
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15
```

Cosa leggere: - default via 10.0.2.2 = il gateway di default è 10.0.2.2 — qualsiasi pacchetto verso una destinazione sconosciuta viene inoltrato lì. È il NAT gateway di VirtualBox. - 10.0.2.0/24 dev eth0 = i pacchetti verso la subnet 10.0.2.0/24 escono da eth0 direttamente (senza passare per il gateway)

Verifica Es. 1 : vedi eth0 con IP 10.0.2.15/24 e gateway 10.0.2.2.

Es. 2 — Ping: testare la raggiungibilità

[fonte: lab HTML, sezione 2 — punto 2]

ping invia messaggi **ICMP ECHO_REQUEST** all'host di destinazione. Se l'host risponde, il percorso di rete tra te e lui è funzionante. È il primo strumento da usare quando una connessione non funziona.

```
# Test verso internet (DNS di Google)
```

```
ping -c 4 8.8.8.8
```

```
# Test verso il gateway VirtualBox
```

```
ping -c 4 10.0.2.2
```

```
# Test verso te stesso (loopback — dovrebbe sempre funzionare)
```

```
ping -c 4 127.0.0.1
```

Anatomia di ping -c 4 8.8.8.8: - ping = invia ICMP ECHO_REQUEST e aspetta ECHO_REPLY - -c 4 = count: invia esattamente 4 pacchetti poi si ferma (senza -c va in loop infinito — ferma con Ctrl+C) - 8.8.8.8 = IP di destinazione

Output atteso (ping -c 4 8.8.8.8):

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=12.3 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=11.8 ms  
...  
4 packets transmitted, 4 received, 0% packet loss
```

Cosa leggere: - time= = latenza round-trip in millisecondi - ttl= = Time To Live — quanti router il pacchetto ha attraversato (parte da 128 o 64, decresce di 1 per ogni router) - 0% packet loss = nessuna perdita → connessione stabile - 100% packet loss = host irraggiungibile

Punto di errore: ping 8.8.8.8 funziona ma ping google.com fallisce → il problema è nel DNS, non nella connettività IP.

Verifica Es. 2 : ping -c 4 8.8.8.8 risponde con 0% packet loss.

Es. 3 — ss: porte in ascolto

ss (socket statistics) mostra lo stato dei socket di rete. È il sostituto moderno di netstat. Permette di vedere quali servizi stanno ascoltando su quali porte — fondamentale per capire la superficie esposta della macchina.

```
ss -tlnp
```

Anatomia di `ss -tlnp`: - `-t` = solo socket TCP (esclude UDP) - `-l` = solo socket in stato LISTEN (in ascolto, non connessioni già stabilite) - `-n` = numeric: non risolvere i numeri di porta in nomi (22 invece di `ssh`) - `-p` = mostra il processo che ha aperto il socket (richiede root per processi altrui)

Output atteso (VM base Debian):

```
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
Process
tcp    LISTEN 0      128    0.0.0.0:22          0.0.0.0:*
users: (("sshd",...))
tcp    LISTEN 0      128    [::]:22            [::]:*
users: (("sshd",...))
```

Solo `sshd` in ascolto sulla porta 22 — è la VM di default senza servizi aggiuntivi.

```
# Installa un servizio e verifica che compaia
sudo apt-get install -y nginx
sudo systemctl start nginx
ss -tlnp # ora vedi anche la porta 80
sudo systemctl stop nginx
```

Connessione con 3A: `ss -tlnp` mostra i demoni in ascolto — la stessa lista che `systemctl list-units --type=service --state=running` mostra da un'altra prospettiva.

Verifica Es. 3 : vedi `sshd` sulla porta 22.

Es. 4 — `/etc/hosts`: risoluzione locale

`/etc/hosts` è il file di risoluzione dei nomi **prima** del DNS. Il sistema lo consulta per primo: se il nome è lì, non chiede al server DNS. Utile per dare nomi a host interni (es. le VM del lab multi-machine di 3E).

```
cat /etc/hosts
```

Output atteso:

```
127.0.0.1      localhost
127.0.1.1      bookworm
::1           localhost ip6-localhost ip6-loopback
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
```

Aggiungi una voce di test:

```
sudo sh -c 'echo "10.0.2.2 gateway.lab" >> /etc/hosts'
ping -c 2 gateway.lab      # usa il nome invece dell'IP
```

Verifica che funzioni:

```
cat /etc/hosts | grep gateway
# → 10.0.2.2 gateway.lab
```

Rimuovi la voce (pulisci):

```
sudo sed -i '/gateway.lab/d' /etc/hosts
```

Punto di errore: modifiche a /etc/hosts hanno effetto immediato — nessun restart. Se il ping con il nome funziona ma prima non funzionava, hai confermato che era un problema DNS (non di routing).

Dove si usa in 3E: quando configuri Client-Router-Server, metti i loro IP in /etc/hosts così puoi usare ping client invece di ping 10.1.1.1.

Verifica Es. 4 : ping gateway.lab risolve e risponde; dopo sed la voce è rimossa.

Es. 5 — dig: query DNS esplicite

dig fa query DNS e mostra la risposta completa. Differenza rispetto a ping hostname: ping risolve il nome silenziosamente; dig ti mostra *come* avviene la risoluzione e qual è la risposta del server DNS.

```
# Installa se non presente
sudo apt-get install -y dnsutils
```

```
# Query A record (IPv4) di google.com
dig google.com A
```

```
# Query verso un DNS specifico (bypassa quello di sistema)
dig @8.8.8.8 google.com A
```

```
# Solo la risposta, senza header
dig +short google.com
```

```
# Reverse lookup: da IP a nome
dig -x 8.8.8.8
```

Anatomia di dig @8.8.8.8 google.com A: - dig = DNS lookup tool -
@8.8.8.8 = usa questo server DNS (non quello di /etc/resolv.conf) -
google.com = nome da risolvere - A = tipo di record (A = IPv4; AAAA = IPv6; MX = mail; NS = nameserver)

Output atteso (parti rilevanti):

```
;; ANSWER SECTION:
google.com.      207      IN      A       142.250.180.46

;; Query time: 12 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
```

Cosa leggere: - ANSWER SECTION = la risposta vera del DNS - 207 = TTL in secondi (tra 207 secondi il record scade dalla cache) - A 142.250.180.46 = l'IP che risolve google.com - Query time = latenza della query DNS

Diagnosi con dig: se ping 8.8.8.8 funziona ma ping google.com fallisce, dig google.com ti dice se il problema è nel DNS (nessuna risposta / SERVFAIL) o nel routing.

Verifica Es. 5 : dig +short google.com restituisce uno o più IP.

Es. 6 — tracepath + tcpdump: percorso e cattura pacchetti

tracepath

[fonte: lab HTML, sezione 2 — punto 3]

tracepath mostra il percorso che i pacchetti fanno per raggiungere una destinazione, hop per hop. Utile quando ping fallisce: ti dice a quale router si interrompe il percorso.

```
# Installa
sudo apt-get install -y iputils-tracepath
```

```
# Traccia il percorso verso 8.8.8.8
tracepath 8.8.8.8
```

Output atteso:

```
1?: [LOCALHOST]                pmtu 1500
1:  10.0.2.2                    0.343ms
1:  10.0.2.2                    0.252ms
2:  [primo router dell'ISP]      ...
...
```

Cosa leggere: ogni riga è un hop (router). Il numero è la posizione nel percorso. Se una riga mostra no reply il router non risponde ai pacchetti di tracepath (normale per molti router).

In 3E: tracepath 10.2.2.2 da Client deve mostrare il salto attraverso Router (10.1.1.254) prima di arrivare al Server.

tcpdump

[fonte: lab HTML, sezione 2 — punto 4]

tcpdump cattura e mostra i pacchetti che transitano su un'interfaccia di rete in tempo reale. È l'equivalente di Wireshark da riga di comando.

Installa

```
sudo apt-get install -y tcpdump
```

Cattura i primi 10 pacchetti su eth0

```
sudo tcpdump -i eth0 -c 10
```

In un'altra sessione (o aspetta che qualcosa generi traffico)

oppure genera traffico tu stesso in background:

```
ping -c 5 8.8.8.8 &
```

```
sudo tcpdump -i eth0 icmp -c 10
```

Anatomia di `sudo tcpdump -i eth0 icmp -c 10`: - sudo = necessario per leggere i pacchetti raw dall'interfaccia - tcpdump = cattura pacchetti - -i eth0 = interfaccia da monitorare - icmp = filtro: cattura solo pacchetti ICMP (quelli di ping) - -c 10 = cattura 10 pacchetti poi si ferma

Output atteso:

```
13:45:01.123456 IP 10.0.2.15 > 8.8.8.8: ICMP echo request, id 42, seq 1
```

```
13:45:01.135678 IP 8.8.8.8 > 10.0.2.15: ICMP echo reply, id 42, seq 1
```

Lettura di una riga: - 13:45:01.123456 = timestamp - IP 10.0.2.15 > 8.8.8.8 = sorgente → destinazione - ICMP echo request = tipo di pacchetto (request = ping inviato, reply = risposta ricevuta)

Filtri utili:

Solo traffico verso/da un host

```
sudo tcpdump -i eth0 host 8.8.8.8 -c 10
```

Solo traffico su porta 22 (SSH)

```
sudo tcpdump -i eth0 port 22 -c 10
```

```
# Salva su file (apribile con Wireshark)
sudo tcpdump -i eth0 -w /tmp/capture.pcap -c 50
```

In 3E: `sudo tcpdump -i eth1` su Router mentre fai `ping 10.2.2.2` da Client — vedi i pacchetti che transitano attraverso il Router.

Verifica Es. 6 : `sudo tcpdump -i eth0 icmp -c 10` mentre esegui `ping -c 5 8.8.8.8` mostra le coppie request/reply.

Riepilogo comandi

Comando	Scopo	Quando usarlo
<code>ip a</code>	Interfacce e indirizzi	Primo controllo: “che IP ho?”
<code>ip r</code>	Tabella di routing	“Dove mandano i pacchetti?”
<code>ping -c N host</code>	Test raggiungibilità	“L’host risponde?”
<code>ss -tlnp</code>	Porte in ascolto	“Quali servizi girano?”
<code>cat /etc/hosts</code>	Risoluzione locale	“Nomi → IP senza DNS”
<code>dig host</code>	Query DNS esplicita	“Come risolve il nome?”
<code>tracert host</code>	Percorso hop-per-hop	“Dove si rompe la rete?”
<code>sudo tcpdump -i eth0</code>	Cattura pacchetti	“Che traffico c’è?”

Dopo il lab

Scrivi i tuoi output negli appunti grezzi (APPUNTI GREZZI/Lab - sysAdm/Appunti_modulo3D.md) e poi esegui `/appunti 3D`.

Collegati

- [[Appunti_modulo3D]]
- [[appunti_modulo3D_networking_base]]
- [[lezione_modulo3D_networking_base]]

Hub: [\[\[master_map_studio\]\]](#) · [\[\[glossario_sysadm\]\]](#) · [\[\[concept_maps\]\]](#) ·
[\[\[troubleshooting_vm\]\]](#) · [\[\[metodo_studio_esami_pratici\]\]](#)